

AI技术应用场景匹配报告

AI技术应用场景匹配报告

摘要

2024—2026 年，AI 在软件工程领域的应用已经从“代码补全工具”扩展为贯穿软件生命周期的智能协作体系。最新研究与产业实践表明，大语言模型、代码智能体、检索增强生成、软件仓库级上下文理解、AIOps 智能体等技术，正在需求分析、系统设计、编码实现、测试验证、发布运维和过程改进中形成可落地的效率提升路径。结合 Dyad 项目的自定义软件过程，本文将 AI 技术与项目中的关键环节进行匹配，识别出 5 个最适合引入 AI 的场景：需求捕获与规格生成、架构设计与 UML/ADR 生成、仓库级编码与代码审查、测试用例生成与自愈测试、发布运维异常检测与根因分析。这些场景与 Dyad “本地运行、快速迭代、以最小可行过程支撑 MVP”的过程定义高度一致，能够在保持轻量化流程的同时提升需求清晰度、架构一致性、实现速度、验证覆盖率和发布后响应能力。

一、2024—2026 年 AI 在软件工程中的应用趋势

从研究侧看，软件工程正在进入 LLM4SE 和 Agentic Software Engineering 阶段。SWE-bench 将真实 GitHub issue 转化为模型需要在完整代码仓库中修复的问题，使代码智能从“生成函数片段”进入“理解仓库、修改多文件、运行测试、提交补丁”的真实工程评价阶段。SWE-agent 进一步说明，智能体不仅需要模型能力，还需要适合智能体使用的 Agent-Computer Interface，包括文件浏览、编辑、测试运行和错误反馈等工具接口。到 2025—2026 年，OpenAI Codex、Claude Code、Cursor、Windsurf/Devin、IBM Bob 等产品都开始强调“并行智能体”“云端沙箱”“代码审查”“长任务规划”“企业级治理”，说明 AI 已经逐步从个人 IDE 插件演化为团队级软件过程参与者。

从生命周期覆盖看，AI 的成熟度存在差异。编码实现阶段最成熟，已有自动补全、自然语言生成代码、代码解释、重构、PR 审查、漏洞分析等能力。测试阶段发展迅速，LLM 可根据代码、需求和覆盖率反馈生成单元测试、接口测试和 UI 测试，并通过生成—执行—修复闭环提高可用性。需求与设计阶段的研究也在升温，系统综述显示 LLM 正被用于需求获取、需求澄清、需求验证、歧义检测和规格文档生成；架构设计方面，LLM 可辅助架构决策分类、设计模式识别、从需求生成架构草图和 UML 模型，但可靠性仍依赖人类评审。运维阶段则从传统 AIOps 的日志聚类 and 异常检测，扩展到基于 LLM 的日志解释、告警摘要、根因分析和自愈建议。

对 Dyad 这类本地 AI 应用构建工具而言，AI 应用的关键不在于把每个步骤完全自动化，而在于把高频、耗时、易返工、上下文密集的活动变成“人类设定目标，AI 生成候选方案，人类评审确认，工具链自动验

证”的闭环。这与 Dyad 的过程裁剪原则一致：需求、设计、实现、测试、发布等核心工程过程必须保留，但不应引入重文档和重审批；对高风险、高影响环节保留规范，对探索性环节允许快速试验。

二、Dyad 软件过程与 AI 场景匹配框架

Dyad 项目定义的软件过程包括立项与需求过程、系统与架构设计过程、工程实现过程、验证与确认过程、发布与部署过程、配置与支持过程、项目管理过程和过程改进过程。其技术栈为 Electron、React、TypeScript，产品目标是通过自然语言快速生成应用原型，并支持界面、代码和交互的持续修改与本地预览。该类项目具有三个特征：第一，需求表达高度自然语言化，用户往往以不完整描述提出目标；第二，架构边界涉及本地运行、AI 对话、工程生成、配置管理、打包发布等多个模块；第三，迭代速度快，测试、发布和问题响应容易成为瓶颈。

因此，AI 技术的匹配标准可设定为四项：一是该环节是否存在大量文本理解、归纳和生成任务；二是该环节是否容易因上下文遗漏导致返工；三是该环节是否有可执行反馈信号，例如测试、lint、构建日志、运行日志；四是该环节是否能够通过人类评审控制风险。按照这四项标准，Dyad 最适合优先落地 AI 的环节不是单纯“替代开发者写代码”，而是围绕需求到交付的关键工件建立智能生成与验证链路。

Dyad 过程环节	可匹配 AI 技术	主要提升目标	风险控制方式
立项与需求过程	需求访谈摘要、用户故事生成、需求一致性检查、RAG 知识检索	降低需求澄清和文档整理时间	人工确认 MVP 范围和优先级
系统与架构设计过程	架构方案生成、UML 类图/时序图生成、ADR 草拟、设计模式推荐	提升架构表达和模块边界清晰度	架构评审、约束清单、反例检查
工程实现过程	仓库级代码智能体、代码补全、重构、代码审查、安全检查	加快功能实现和缺陷修复	测试、lint、PR review、最小变更原则
验证与确认过程	单元测试生成、覆盖率驱动补测、自愈 UI 测试、缺陷预测	提高测试覆盖率并降低维护成本	生成—执行—修复闭环，人工审查关键断言
发布与运维支持过程	日志异常检测、告警摘要、根因分析、发布风险预测	缩短问题定位和发布后响应时间	只推荐不自动破坏性执行，保留回滚机制

三、关键应用场景一：AI 辅助需求捕获、文档生成与一致性验证

Dyad 的需求过程目标是明确项目要解决的问题、服务对象、核心能力和 MVP 范围。由于 Dyad 面向“用自然语言快速生成应用”的场景，需求往往来自用户一句话描述、反馈 issue、社区讨论或功能设想，原始输入存在模糊、重复、遗漏非功能需求等问题。2024—2026 年的需求工程研究表明，LLM 已经被广泛用于需求获取、需求分类、需求验证和规格文档生成。系统综述发现，需求工程中的 LLM 应用主要集中在需求获取与验证，常用方式包括零样本提示、少样本提示、交互式澄清和 RAG。

在 Dyad 中，可建立“需求智能整理器”：输入用户访谈、issue、产品想法或课堂作业中的自然语言描述，AI 自动输出用户画像、用户故事、功能需求、非功能需求、约束条件、验收标准和优先级建议。例如，对于“自然语言生成应用原型”这一核心能力，AI 不仅生成“用户可以输入应用描述并生成项目”的功能需求，还应补充性能、稳定性、安全、隐私、本地运行、失败恢复等非功能需求。随后，AI 可执行一致性检查：识别需求之间的冲突，如“完全本地运行”与“依赖云端模型服务”的矛盾；识别不可测试需求，如“界面要好看”应转化为可验证的设计规范或用户满意度指标；识别 MVP 范围膨胀，如数据库集成、导入导出、发布准备是否属于当前迭代。

该场景的效率价值主要体现在三个方面。第一，缩短需求文档起草时间，使轻量过程也能保留必要工件。第二，减少后续设计和编码阶段因需求不清导致的返工。第三，把需求与测试验收提前关联，为后续测试用例生成提供输入。风险在于 LLM 可能“补全”并不存在的业务目标，因此应采用人机协同方式：AI 负责生成候选规格和冲突清单，产品或开发负责人确认最终需求基线。

四、关键应用场景二：AI 驱动架构设计、UML 模型生成与设计模式推荐

Dyad 的系统与架构设计过程需要划分用户界面模块、AI 对话与提示词处理模块、工程生成模块、配置管理模块、打包发布模块，并明确本地运行架构、关键数据流、控制流和技术选型。软件架构相关研究显示，LLM 已开始应用于架构决策分类、设计模式检测、从需求生成架构设计、生成参考架构等任务。2026 年关于从自然语言需求生成 UML 类图的研究还开始使用 LLM-as-a-Judge 与人工评审结合的双重验证框架，说明模型生成设计工件已经具备探索价值，但仍不能脱离人工确认。

在 Dyad 中，该场景可落地为“架构设计助手”。输入需求列表和技术约束后，AI 生成模块分解、依赖关系、接口草案、主要数据对象、异常流程和 ADR。对于 Electron + React + TypeScript 项目，AI 可建议主进程与渲染进程的职责边界、IPC 接口设计、本地文件读写权限、项目模板生成流程、模型调用抽象层、配置持久化策略等。进一步地，AI 可将设计结果生成 UML 类图、时序图或 C4 风格上下文图，用于团队讨论和文档沉淀。

设计模式推荐是该场景的另一个价值点。例如，AI 对话与工具调用可采用命令模式或策略模式，将不同模型提供商、提示词模板、工程生成器封装为可替换策略；配置管理可采用仓储模式隔离存储实现；应用生成过程可采用流水线或责任链模式处理“解析需求—生成文件—安装依赖—运行预览—修复错误”的

步骤。AI 的作用不是机械套用模式，而是根据模块变化频率、可测试性、扩展性和本地运行约束提出备选方案，并解释取舍。

该场景的效率收益在于减少架构文档从零开始编写的成本，使轻量项目也能形成可追溯的设计依据。风险在于 LLM 可能给出看似完整但不可实现的架构，因此应强制要求输出“假设、约束、未决问题、替代方案和验证办法”，并将架构建议转化为可执行的原型验证任务。

五、关键应用场景三：仓库级代码生成、智能代码审查与重构

工程实现过程是当前 AI 工具最成熟的落地点。SWE-bench 和 SWE-agent 的出现说明，真实软件工作任务要求模型理解整个仓库、定位相关文件、修改多处代码、运行测试并根据反馈修复。OpenAI Codex、Claude Code、Cursor、Windsurf/Devin 和 IBM Bob 的最新实践都强调仓库级上下文、工具调用、并行任务、代码审查和企业治理。OpenAI Codex 已从研究预览发展到团队级可用工具，支持云端任务、PR 审查、SDK 和 CI/CD 集成；Anthropic 的 Claude Code 强调将智能体嵌入终端、仓库和开发者 workflow，并通过权限控制、上下文管理和可审查操作降低自动化风险；Cursor 3 和 Cursor 2.0 强调本地/云端智能体协作与多智能体并行；Windsurf 的 SWE-1 系列则面向真实开发 workflow 优化工具调用推理。

对 Dyad 来说，AI 编码不应只用于生成零散函数，而应嵌入工程实现流程。典型用法包括：根据需求和架构任务生成初始实现；为 React 组件补齐状态管理和边界情况；为 Electron 主进程实现本地文件操作和权限检查；根据 TypeScript 类型错误和构建日志自动修复；对重复逻辑进行重构；为 PR 自动生成变更摘要、风险点和审查建议。

智能代码审查尤其适合 Dyad。AI 应检查的不只是语法，还包括跨文件行为一致性、异常处理、用户输入校验、潜在安全问题、是否破坏本地运行假设、是否引入不必要依赖、是否缺少测试。对于 AI 应用构建工具，提示词注入、文件系统访问、模型输出写入项目文件等场景都存在安全风险，因此可引入类似 Codex Security 的思路，让 AI 基于项目上下文进行威胁建模和漏洞验证。

该场景的效率收益最高，但也最容易产生质量幻觉。落地时应采用“AI 生成补丁，人类审查意图，自动化工具验证结果”的机制。所有 AI 生成代码必须通过 TypeScript 类型检查、lint、单元测试、构建和最小人工 review。对于核心生成器、文件系统操作、发布打包等高影响模块，应限制 AI 一次变更范围，避免大规模重写。

六、关键应用场景四：AI 生成测试用例、自愈测试脚本与预测性缺陷检测

Dyad 的验证与确认过程包括功能测试、非功能验证、实际场景验证和用户反馈收集。传统测试在快速迭代项目中容易滞后，尤其是 UI、工程生成结果、跨平台兼容和本地预览流程。2024 年 IEEE TSE 关于

LLM 软件测试的综述分析了大量研究，指出 LLM 已用于测试用例生成、测试输入生成、断言生成、测试修复和缺陷检测。CoverUp 等研究进一步将覆盖率反馈引入 LLM 测试生成，使模型根据未覆盖代码继续补测，而不是一次性生成测试后停止。

在 Dyad 中，可优先建设“AI 测试助手”。对于工程生成模块，AI 根据需求和代码生成单元测试，覆盖正常生成、非法输入、依赖安装失败、文件冲突、模板缺失等情况。对于 React UI，AI 根据组件行为生成交互测试，验证用户输入需求、生成应用、修改应用、运行预览等核心流程。对于 Electron 跨平台能力，AI 根据发布目标生成检查清单，如 Windows/macOS/Linux 下文件路径、权限、打包资源和本地存储差异。

自愈测试脚本适合用于 UI 自动化测试。Dyad 界面在迭代中可能频繁调整按钮文案、DOM 结构和组件层级，传统选择器容易失效。AI 可结合 DOM、可访问性树、视觉线索和历史定位信息，在测试失败时推荐新的定位方式，并生成修复补丁。但自愈不应无条件自动接受，因为它可能把真实 UI 缺陷误判为选择器变化。较合理的做法是：测试失败后 AI 给出“失败原因分类、建议修复、是否可能是真实缺陷”，由开发者确认后合并。

预测性缺陷检测可结合代码变更规模、历史缺陷、复杂度、测试覆盖率和 AI 审查结果，为每个 PR 标注风险等级。对 Dyad 来说，高风险变更包括工程生成器、模型调用适配层、本地文件写入、打包发布脚本和配置迁移逻辑。AI 可在合并前提示需要增加的测试类型，从而把质量控制前移。

七、关键应用场景五：AI 异常检测、根因分析与发布后支持

Dyad 的发布与部署过程要求生成安装包、准备发布说明、整理版本变更、提供下载方式，并对发布后的问题快速响应。配置与支持过程还包括版本管理、缺陷跟踪、文档维护、开发环境管理和社区支持。随着 AI 应用本地化和跨平台运行，发布后的问题往往来自安装环境、依赖版本、模型服务配置、系统权限、打包资源路径、用户输入导致的生成失败等复杂因素。

AIOps 研究在 2024—2026 年快速发展。Microsoft AIOpsLab 提出面向自治云的智能体评测框架，通过故障注入、云环境接口和智能体交互来评估定位与恢复能力。AetherLog 将 LLM 与知识图谱结合用于日志级根因分析。IBM Research 则提出面向 SRE 的 LLM 辅助异常检测服务，用通用 API 处理工业时间序列数据，支持对复杂数据流的主动异常识别。这些成果说明，运维 AI 的重点已经从单点告警分类走向“日志、指标、拓扑、历史问题、代码上下文”的综合分析。

在 Dyad 中，可建立轻量级“发布与支持智能体”。发布前，AI 根据 commit、PR、issue 和测试结果生成 release note、升级风险清单和回滚建议。发布后，AI 收集用户日志、错误栈、系统信息和操作步骤，自动进行脱敏、聚类 and 摘要，判断是否属于已知问题，并给出可能根因。例如，若多个 Windows 用户反馈应用生成后预览失败，AI 可关联日志中的路径、权限、依赖安装失败信息和最近发布变更，提示问题可能来自打包资源路径或 shell 执行策略差异。

智能扩容与优化在 Dyad 中可理解为本地资源与模型调用优化，而非传统云服务自动扩容。AI 可分析生成任务耗时、模型调用失败率、缓存命中率、依赖安装耗时和预览启动时间，提出优化建议，如缓存模板、减少重复模型调用、拆分长提示词、增加失败重试和降级策略。该场景能显著降低发布后支持成本，并为过程改进提供数据。

八、实施优先级与落地路线

综合收益、风险和实现成本，Dyad 可按三个阶段引入 AI。第一阶段优先落地需求整理、代码审查和测试生成，因为这些环节输入输出清晰，且容易通过人工评审与自动化验证控制风险。第二阶段引入架构设计助手和 UML/ADR 生成，使需求到实现之间形成更稳定的设计工件。第三阶段建设发布后日志分析、异常聚类 and 根因分析能力，把用户反馈纳入过程改进闭环。

短期可交付成果包括：需求模板生成器、AI 需求一致性检查清单、PR 审查提示词、单元测试生成流程、发布说明生成器。中期可交付成果包括：架构设计助手、UML 图生成、覆盖率驱动补测、自愈 UI 测试建议。长期可交付成果包括：仓库级智能体任务队列、发布风险预测、日志根因分析、面向过程改进的度量看板。

为了避免 AI 引入新的不确定性，Dyad 应建立四条基本治理规则。第一，AI 生成的需求、设计和代码都必须保留来源、输入和评审记录。第二，凡是影响文件系统、发布打包、安全边界和用户数据的变更，必须人工确认。第三，AI 生成测试不能只追求覆盖率，还要审查断言是否真正验证需求。第四，AI 参与过程改进时应基于真实数据，包括缺陷来源、返工原因、测试失败类型和发布后反馈，而不是只生成泛化建议。

九、结论

AI 对软件工程的提升不再局限于“写代码更快”，而是正在改变软件过程中的工件生产、上下文管理、质量验证和运维响应方式。结合 Dyad 的轻量化、自适应、快速迭代过程，最值得优先引入 AI 的 5 个关键环节是：需求捕获与一致性验证、架构设计与 UML/ADR 生成、仓库级编码与代码审查、测试生成与自愈测试、发布运维异常检测与根因分析。这些环节共同构成从需求到反馈的智能闭环：需求更清楚，设计更可追溯，实现更快，测试更充分，发布后问题定位更及时。

但 AI 的正确定位应是“可验证的工程协作者”，而不是完全替代软件过程。对于 Dyad 这样的 AI 应用构建工具，最佳策略是让 AI 承担高频、重复、上下文密集的生成与分析任务，让人类保留目标设定、架构取舍、安全判断和最终验收权。这样既符合最小可行过程的思想，也能在不引入重流程负担的前提下显著提升软件开发效率和质量。

参考文献

- [1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, “SWE–bench: Can Language Models Resolve Real–World GitHub Issues?” in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. <https://openreview.net/forum?id=VTF8yNQM66>
- [2] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “SWE–agent: Agent–Computer Interfaces Enable Automated Software Engineering,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024. <https://arxiv.org/abs/2405.15793>
- [3] M. Shetty et al., “Building AI Agents for Autonomous Clouds: Challenges and Design Principles,” in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, 2024. <https://arxiv.org/abs/2407.12165>
- [4] T. Cui, R. Fu, C. Liu, Y. Ji, W. Gu, S. Zhang, Y. Sun, and D. Pei, “AetherLog: Log–based Root Cause Analysis by Integrating Large Language Models with Knowledge Graphs,” in *IEEE International Symposium on Software Reliability Engineering (ISSRE)*, pp. 49–60, 2025. <https://dblp.org/rec/conf/issre/CuiFLJGZSP25>
- [5] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, “Software Testing With Large Language Models: Survey, Landscape, and Vision,” *IEEE Transactions on Software Engineering*, vol. 50, no. 4, pp. 911–936, 2024. <https://doi.org/10.1109/TSE.2024.3368208>
- [6] B. Pham, “CoverUp: Coverage–Guided LLM–Based Test Generation,” arXiv:2403.16218, 2024. <https://arxiv.org/abs/2403.16218>
- [7] A. Ahmad et al., “Research Directions for Using LLM in Software Requirement Engineering: A Systematic Review,” *Frontiers in Computer Science*, 2025. <https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2025.1519437/full>
- [8] D. Giray, B. Tekinerdogan, and I. N. Athanasiadis, “Software Architecture Meets LLMs: A Systematic Literature Review,” arXiv:2505.16697, 2025. <https://arxiv.org/abs/2505.16697>
- [9] M. Genero et al., “Class Model Generation from Requirements using Large Language Models,” arXiv:2603.09100, 2026. <https://arxiv.org/abs/2603.09100>
- [10] OpenAI, “Introducing Codex,” 2025. <https://openai.com/index/introducing-codex/>
- [11] OpenAI, “Codex is now generally available,” 2025. <https://openai.com/index/codex-now-generally-available/>
- [12] Anthropic, “Claude Code: Best practices for agentic coding,” 2025. <https://www.anthropic.com/engineering/claude-code-best-practices>

- [13] Cursor, “Introducing Cursor 2.0 and Composer,” 2025. <https://cursor.com/blog/2-0>
- [14] Cursor, “Meet the new Cursor,” 2026. <https://cursor.com/blog/cursor-3>
- [15] Windsurf/Devin, “SWE-1: Our First Frontier Models,” 2025.
<https://windsurf.com/blog/windsurf-wave-9-swe-1>
- [16] IBM, “AI coding agent for enterprises,” 2026. <https://www.ibm.com/products/ai-coding-agent>
- [17] IBM Research, “LLM Assisted Anomaly Detection Service for Site Reliability Engineers: Enhancing Cloud Infrastructure Resilience,” AAAI 2025.
<https://research.ibm.com/publications/llm-assisted-anomaly-detection-service-for-site-reliability-engineers-enhancing-cloud-infrastructure-resilience>